

# **Aggiornamenti OTA con Secure-Boot e Flash encryption sul dispositivo embedded ESP32**

UniVR - Dipartimento di Informatica

**Fabio Irimie**

Tesi di laurea 2025/2026

# Indice

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                          | <b>2</b>  |
| <b>2</b> | <b>Cenni teorici</b>                         | <b>2</b>  |
| 2.1      | Aggiornamenti OTA (Over The Air) . . . . .   | 2         |
| 2.1.1    | Partizione OTA Data . . . . .                | 3         |
| 2.1.2    | App rollback . . . . .                       | 3         |
| 2.2      | eFuse . . . . .                              | 4         |
| 2.3      | Secure Boot . . . . .                        | 5         |
| 2.3.1    | Vantaggi . . . . .                           | 5         |
| 2.3.2    | Formato del signature block . . . . .        | 6         |
| 2.3.3    | Secure padding . . . . .                     | 7         |
| 2.3.4    | Verifica di un'immagine . . . . .            | 7         |
| 2.3.5    | Processo di Secure Boot . . . . .            | 8         |
| 2.4      | Flash Encryption . . . . .                   | 8         |
| 2.4.1    | eFuse rilevanti . . . . .                    | 9         |
| 2.4.2    | Processo della Flash Encryption . . . . .    | 9         |
| 2.4.3    | Modalità di sviluppo e di rilascio . . . . . | 10        |
| <b>3</b> | <b>Implementazione</b>                       | <b>11</b> |
| 3.1      | Aggiornamenti OTA . . . . .                  | 11        |
| 3.1.1    | Configurazione del progetto . . . . .        | 11        |
| 3.1.2    | Connessione Wi-Fi . . . . .                  | 13        |
| 3.1.3    | Aggiornamento OTA . . . . .                  | 14        |
| 3.1.4    | Applicazione principale . . . . .            | 15        |
| 3.2      | Flash Encryption . . . . .                   | 15        |
| 3.2.1    | Configurazione del progetto . . . . .        | 15        |
| 3.3      | Secure boot . . . . .                        | 17        |
| 3.3.1    | Configurazione del progetto . . . . .        | 17        |
| <b>4</b> | <b>Analisi dei risultati</b>                 | <b>18</b> |
| 4.1      | Verifica dello stato di sicurezza . . . . .  | 18        |
| 4.1.1    | Controllo dello stato degli eFuse . . . . .  | 19        |
| 4.1.2    | Controllo dei log della console . . . . .    | 19        |
| 4.2      | Test di aggiornamento OTA . . . . .          | 20        |
| <b>5</b> | <b>Conclusioni</b>                           | <b>21</b> |
|          | <b>Bibliografia</b>                          | <b>21</b> |

# 1 Introduzione

Questo progetto consiste nell'implementazione di un sistema che permetta di aggiornare il firmware dell'ESP32 da remoto (Over The Air) tramite Wi-Fi. L'obiettivo principale è quello di attivare le funzionalità di sicurezza del micro-controllore in modo da proteggere il dispositivo da accessi non autorizzati. Le funzionalità di sicurezza includono:

- **Secure OTA:** Garantisce che il nuovo firmware sia autentico e non compromesso
- **Secure Boot:** Impedisce l'esecuzione di firmware non autorizzato
- **Flash Encryption:** Protegge i dati memorizzati nella memoria flash del dispositivo

## 2 Cenni teorici

### 2.1 Aggiornamenti OTA (Over The Air)

Gli aggiornamenti OTA permettono di aggiornare il firmware del dispositivo durante la sua normale esecuzione, senza la necessità di collegarlo fisicamente a un computer. Le modalità di aggiornamento si distinguono in base alla vulnerabilità del sistema:

- **Modalità sicura:** L'aggiornamento di alcune partizioni è resiliente, cioè garantisce l'operabilità del dispositivo anche in caso di perdita di alimentazione o di errore durante l'aggiornamento. Solo il seguente tipo di partizione supporta la modalità sicura:
  - **Application:** OTA configura la partition table in modo da avere due partizioni per l'aggiornamento (`ota_0` e `ota_1`) e una partizione per lo stato di boot (`ota_data`). Durante l'aggiornamento il nuovo firmware viene scritto nella partizione OTA attualmente non selezionata per il boot. Una volta completato l'aggiornamento, la partizione `ota_data` viene aggiornata per indicare che la partizione OTA appena scritta deve essere utilizzata al boot successivo. Se la partizione `ota_data` non contiene alcun dato il dispositivo esegue il boot dalla partizione `factory`.

La partition table con due partizioni OTA è la seguente:

```
1 # ESP-IDF Partition Table
2 # Name, Type, SubType, Offset, Size, Flags
3 nvs, data, nvs, 0x9000, 0x4000,
4 otadata, data, ota, 0xd000, 0x2000,
5 phy_init, data, phy, 0xf000, 0x1000,
6 factory, app, factory, 0x10000, 1M,
7 ota_0, app, ota_0, 0x110000, 1M,
8 ota_1, app, ota_1, 0x210000, 1M,
9
```

- **Modalità non sicura:** L'aggiornamento di alcune partizioni è vulnerabile, cioè in caso di perdita di alimentazione o di errore durante l'aggiornamento il dispositivo potrebbe non essere più operabile. Una partizione

temporanea riceve i dati della nuova immagine e, una volta completato il trasferimento, l'immagine viene copiata nella partizione di destinazione. Se l'operazione di copia viene interrotta potrebbero verificarsi problemi di boot. Le partizioni che supportano la modalità non sicura sono:

- **Bootloader**
- **Partition Table**
- **Partizioni data** (ad esempio NVS, FAT, ecc...)

### 2.1.1 Partizione OTA Data

Al primo avvio del dispositivo la partizione `ota_data` deve essere vuota (tutti i byte a `0xFF`) in modo da far eseguire il boot dall'applicazione nella partizione `factory`. Se l'applicazione in `factory` non è presente viene eseguito il boot della prima partizione OTA disponibile (di solito `ota_0`).

Dopo il primo aggiornamento OTA, la partizione `ota_data` viene aggiornata per indicare quale partizione OTA deve essere utilizzata al successivo boot. La dimensione di `ota_data` è di due settori (`0x2000` bytes = `8192` bytes) in modo da evitare errori mentre si scrive la partizione. I due settori sono cancellati indipendentemente e scritti con gli stessi dati. In questo modo se i dati dei due settori non coincidono viene usato un counter per determinare quale settore è stato scritto più recentemente.

### 2.1.2 App rollback

L'obiettivo dell'app rollback è quello di tenere il funzionante il dispositivo dopo un aggiornamento e permette di tornare alla versione precedente del firmware se la nuova versione non funziona correttamente (solo le partizioni OTA possono effettuare il rollback). Dopo un aggiornamento OTA con rollback attivo si hanno le seguenti possibilità:

- Se l'app funziona bene `esp_ota_mark_app_valid_cancel_rollback()` imposta lo stato dell'applicazione a `ESP_OTA_IMG_VALID`.
- Se l'app non funziona correttamente il dispositivo esegue il rollback alla versione precedente e `esp_ota_mark_app_invalid_rollback()` imposta lo stato dell'applicazione a `ESP_OTA_IMG_INVALID`.
- Se l'impostazione `CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE` è abilitata e viene effettuato un reset, allora viene effettuato un rollback senza chiamare nessuna funzione nell'applicazione. Questa opzione permette di intercettare la prima esecuzione di una nuova applicazione per confermare che funzioni correttamente.

Gli stati che controllano il processo di selezione dell'applicazione sono:

| Stato                      | Restrizioni sulla nuova app                                                                                                                                                                                     |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ESP_OTA_IMG_VALID          | Nessuna restrizione. Verrà selezionata                                                                                                                                                                          |
| ESP_OTA_IMG_UNDEFINED      | Nessuna restrizione. Verrà selezionata                                                                                                                                                                          |
| ESP_OTA_IMG_INVALID        | Non verrà selezionata                                                                                                                                                                                           |
| ESP_OTA_IMG_ABORTED        | Non verrà selezionata                                                                                                                                                                                           |
| ESP_OTA_IMG_NEW            | Se l'opzione <code>CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE</code> è abilitata, l'app verrà selezionata solo una volta. Nel bootloader lo stato viene subito impostato a <code>ESP_OTA_IMG_PENDING_VERIFY</code> . |
| ESP_OTA_IMG_PENDING_VERIFY | Se l'opzione <code>CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE</code> è abilitata, l'app non verrà selezionata. Nel bootloader lo stato viene impostato a <code>ESP_OTA_IMG_ABORTED</code> .                          |

Tabella 1: Stati dell'applicazione OTA

L'impostazione di questi stati avviene nei seguenti casi:

- `ESP_OTA_IMG_VALID`: impostato dalla funzione `esp_ota_mark_app_valid_cancel_rollback()`.
- `ESP_OTA_IMG_UNDEFINED`: impostato dalla funzione `esp_ota_set_boot_partition()` se l'impostazione `CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE` è disabilitata.
- `ESP_OTA_IMG_NEW`: impostato dalla funzione `esp_ota_set_boot_partition()` se l'impostazione `CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE` è abilitata.
- `ESP_OTA_IMG_INVALID`: impostato dalla funzione `esp_ota_mark_app_invalid_rollback()` o `esp_ota_mark_app_invalid_rollback_and_reboot()`.
- `ESP_OTA_IMG_ABORTED`: impostato se l'operabilità dell'applicazione non è stata confermata e avviene un reboot quando l'impostazione `CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE` è abilitata.
- `ESP_OTA_IMG_PENDING_VERIFY`: impostato nel bootloader se l'impostazione `CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE` è abilitata e l'applicazione selezionata è nello stato `ESP_OTA_IMG_NEW`.

## 2.2 eFuse

Gli **eFuse** (electronic fuses) sono memorie non volatili che possono essere programmate (burned) una sola volta. Gli **eFuse** sono campi da **un bit** che può essere impostato a 1, ma non può essere riportato a 0 e servono per memorizzare valori di sistema o dell'utente. I bit sono raggruppati in 4 blocchi da 256 bit e ogni blocco è suddiviso in 8 registri da 32 bit. Alcuni blocchi sono riservati per i valori di sistema e altri sono liberi per l'uso dell'utente. I blocchi degli eFuse sono divisi in:

- EFUSE.BLK0: è interamente riservato ai valori di sistema
- EFUSE.BLK1: è utilizzato per le chiavi della Flash Encryption se attivata, altrimenti può essere usato dall'utente
- EFUSE.BLK2: è utilizzato per la chiave del Secure Boot se attivo, altrimenti può essere usato dall'utente
- EFUSE.BLK3: può essere riservato per memorizzare un indirizzo MAC personalizzato oppure può essere usato dall'utente. Alcuni bit sono già usati in ESP-IDF.

Non tutti i bit possono essere utilizzati.

## 2.3 Secure Boot

Il Secure Boot protegge il dispositivo dall'esecuzione di firmware non autorizzato verificando che ogni software (second stage bootloader e ogni applicazione) che viene eseguito sia firmato. Il first stage bootloader non deve essere firmato in quanto è memorizzato in una memoria ROM (di sola lettura). La firma è una coppia di chiavi (privata e pubblica) generata tramite l'algoritmo RSA.

- La chiave privata viene utilizzata per firmare il software prima che venga caricato nel dispositivo.
- La chiave pubblica viene memorizzata nel dispositivo e viene utilizzata per verificare la firma del software prima che venga eseguito.

In breve, il processo di Secure Boot funziona come segue:

1. Il first stage bootloader carica il second stage bootloader e ne verifica la firma. Solo se la verifica va a buon fine il second stage bootloader viene eseguito.
2. Il second stage bootloader carica l'applicazione e ne verifica la firma. Solo se la verifica va a buon fine l'applicazione viene eseguita.

### 2.3.1 Vantaggi

I vantaggi del Secure Boot sono:

- La chiave pubblica è memorizzata sul dispositivo, mentre quella privata è tenuta in un posto sicuro e non viene mai usata dal dispositivo.
- Solo una chiave pubblica può essere generata e memorizzata nel chip durante la fase di produzione.
- Viene usato lo stesso formato di immagine del firmware e della firma sia per il second stage bootloader che per le applicazioni.
- Nessun dato segreto viene memorizzato nel dispositivo.

### 2.3.2 Formato del signature block

Il signature block è una struttura che contiene la firma del software. Il blocco inizia su un confine allineato a 4 KB e ha un proprio settore di flash, cioè 4096 byte. La firma viene calcolata su **tutti i byte** dell'immagine, inclusi i byte di "secure padding" (vedi capitolo 2.3.3). Ogni signature block contiene la firma dell'immagine a cui appartiene insieme alla chiave pubblica RSA usata per la verifica della firma. Il formato del signature block è il seguente:

| Offset | Dimensione (byte) | Descrizione                                                                                                                                                                                                  |
|--------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0      | 1                 | Byte magico (deve essere 0xE7)                                                                                                                                                                               |
| 1      | 1                 | Byte della versione, per Secure Boot v2 è 0x02                                                                                                                                                               |
| 2      | 2                 | Byte di padding. Riservati e devono essere a 0                                                                                                                                                               |
| 4      | 32                | Hash SHA-256 del contenuto dell'immagine, senza considerare il signature block                                                                                                                               |
| 36     | 384               | Modulo RSA usato per la verifica della firma. (valore "n" nella specifica RFC8017)                                                                                                                           |
| 420    | 4                 | Esponente pubblico RSA usato per la verifica della firma. (valore "e" nella specifica RFC8017)                                                                                                               |
| 424    | 384               | Valore precalcolato di "R", derivato da "n", usato per l'algoritmo di moltiplicazione di Montgomery                                                                                                          |
| 808    | 4                 | Valore precalcolato di "M'", derivato da "n", usato per l'algoritmo di moltiplicazione di Montgomery                                                                                                         |
| 812    | 384               | Firma RSA-PSS (sezione 8.1.1 della RFC8017) del contenuto dell'immagine, calcolata usando i seguenti parametri PSS: hash SHA-256, funzione MGF1, lunghezza del salt 32 bytes, campo trailer di default 0xBC. |
| 1196   | 4                 | CRC32 (checksum) dei precedenti 1196 bytes                                                                                                                                                                   |
| 1200   | 16                | Byte di padding a 0 per arrivare a lunghezza 1216 bytes                                                                                                                                                      |

Tabella 2: Formato del signature block

Lo spazio rimanente dopo il signature block ( $4096 - 1216 = 2880$  bytes) è memoria flash cancellata, cioè tutti i byte a 0xFF, che può essere usata per scrivere altri signature block dopo il precedente.

Un signature block è valido se soddisfa entrambe le condizioni:

- Il byte magico è corretto (0xE7)

- La checksum CRC32 è corretta

altrimenti viene considerato non valido.

### 2.3.3 Secure padding

Il secure padding è un'area di memoria che viene aggiunta alla fine di ogni immagine del firmware per allineare l'immagine al confine della dimensione di pagina della flash MMU (Memory Management Unit con dimensione default di 64KB). Questo viene fatto per assicurare che soltanto contenuti verificati vengano mappati nella memoria indirizzabile del dispositivo. La firma dell'immagine viene calcolata dopo aver aggiunto il secure padding e solo dopo viene aggiunto alla fine il signature block (4KB).

### 2.3.4 Verifica di un'immagine

Un esempio di applicazione firmata è il seguente:

| Offset | Dimensione (KB) | Descrizione                                                         |
|--------|-----------------|---------------------------------------------------------------------|
| 0      | 580             | Contenuto di esempio di un'applicazione non firmata                 |
| 580    | 60              | Secure padding per allineare l'immagine al prossimo confine di 64KB |
| 640    | 4               | Signature block                                                     |

Tabella 3: Esempio di applicazione firmata

L'immagine dell'applicazione inizia sempre al prossimo confine di pagina della flash MMU, di default 64KB, e quindi lo spazio rimanente dopo il signature block può essere utilizzato per memorizzare altre partizioni di dati, ad esempio `nvs`.

Un'immagine è verificata se la chiave pubblica memorizzata in qualsiasi signature block è valida per quel dispositivo, e se la firma RSA-PSS nel signature block coincide con la firma calcolata per i dati dell'immagine letti dalla flash. La verifica dell'immagine non viene effettuata soltanto ad ogni boot, ma anche dopo ogni aggiornamento OTA. Se la verifica della nuova immagine ottenuta tramite OTA fallisce, il bootloader cercherà un'altra immagine valida da eseguire. I passaggi per verificare un'immagine sono i seguenti:

1. Confrontare l'hash SHA-256 della chiave pubblica memorizzata nel signature block del bootloader con quello memorizzato negli `eFuse` del dispositivo. Se non coincidono, l'immagine non è valida.
2. Generare l'hash dell'immagine dell'applicazione e confrontarlo con l'hash memorizzato nel signature block dell'immagine. Se non coincidono, l'immagine non è valida.
3. Usare la chiave pubblica per verificare la firma dell'immagine del bootloader usando RSA-PSS (sezione 8.1.2 della RFC8017) per confrontarla con l'hash dell'immagine calcolato nel passo 2.



### 2.3.5 Processo di Secure Boot

I passi eseguiti durante il processo di Secure Boot sono i seguenti:

1. All'avvio, il codice salvato in ROM (first stage bootloader) controlla il bit di Secure Boot v2 negli **eFuse** del dispositivo. Se il bit è disabilitato, il dispositivo esegue il boot normalmente, altrimenti procede al passo successivo.
2. Il codice in ROM verifica il signature block del bootloader (vedi 2.3.2). Se fallisce il processo di boot viene interrotto.
3. Il codice in ROM verifica l'immagine del bootloader usando i dati dell'immagine, i signature block corrispondenti e gli **eFuse** (vedi 2.3.4). Se la verifica fallisce il processo di boot viene interrotto.
4. Il codice in ROM esegue il bootloader.
5. Il bootloader verifica il signature block dell'immagine dell'applicazione. Se fallisce il processo di boot viene interrotto.
6. Il bootloader verifica l'immagine dell'applicazione usando i dati dell'immagine, i signature block corrispondenti e gli **eFuse**. Se la verifica fallisce il processo di boot viene interrotto, ma se viene trovata un'altra immagine, allora il bootloader proverà a verificare quell'immagine tornando ad eseguire i punti dal 5 al 7. Questo viene ripetuto finché non viene trovata un'immagine valida o non ci sono più immagini da verificare.
7. Il bootloader esegue l'immagine dell'applicazione verificata.

## 2.4 Flash Encryption

La Flash Encryption permette di crittografare la memoria flash del dispositivo. Una volta attivata, il firmware viene flashato sottoforma di testo non cifrato che verrà crittografato al primo boot. Una volta attivata, le seguenti partizioni vengono crittografate di default:

- Second stage bootloader
- Partition table
- Partizione OTA data
- Partizioni di tipo **app**

Altre partizioni si possono crittografare condizionalmente:

- Partizioni con la flag **encrypted** impostata nella partition table
- Il digest (hash) del bootloader di Secure Boot se il Secure Boot è abilitato

### 2.4.1 eFuse rilevanti

I seguenti eFuse sono rilevanti per la Flash Encryption:

- **CODING\_SCHEME** (2 bit): definisce il numero di bit del **block1** usati per derivare la chiave di crittografia finale. I possibili valori sono:
  - 0: per 256 bits
  - 1: per 192 bits
  - 2: per 128 bits

Il metodo di derivazione è basato sul valore dell'eFuse **FLASH\_CRYPT\_CONFIG**.

- **flash\_encryption** (**block1** 256 bit): contiene la chiave di crittografia
- **FLASH\_CRYPT\_CONFIG** (4 bit): controlla il processo di crittografia
- **DISABLE\_DL\_ENCRYPT** (1 bit): se abilitato disattiva la flash encryption mentre il dispositivo viene eseguito in modalità Firmware Download
- **DISABLE\_DL\_DECRYPT** (1 bit): se abilitato disattiva la flash decryption mentre il dispositivo viene eseguito in modalità UART Firmware Download
- **FLASH\_CRYPT\_CNT** (7 bit): è un numero del tipo  $2^n$  indica se i contenuti della memoria flash sono stati crittografati.
  - Se un numero dispari di bit sono impostati a 1, indica che i contenuti della memoria flash sono crittografati e devono essere decrittografati durante la lettura.
  - Se un numero pari di bit sono impostati a 1, indica che i contenuti della memoria flash non sono crittografati.

Ad ogni aggiornamento della memoria flash con dati non crittografati e con l'esecuzione della crittografia il bit più significativo di **FLASH\_CRYPT\_CNT** viene impostato a 1.

### 2.4.2 Processo della Flash Encryption

Supponendo che i valori degli eFuse siano nello stato iniziale e che il second stage bootloader sia stato compilato per supportare la flash encryption, il processo di crittografia è il seguente:

1. Al primo avvio, tutti i dati nella memoria flash non sono crittografati (testo in chiaro) e il first stage bootloader carica il second stage bootloader.
2. Il second stage bootloader controlla il valore dell'eFuse **FLASH\_CRYPT\_CNT**. Poichè tutti i bit sono a 0 (stato di default), cioè un numero pari di bit impostati, il bootloader configura e attiva il *flash encryption block*. Viene anche impostato il valore dell'eFuse **FLASH\_CRYPT\_CONFIG** a 0xF
3. Il second stage bootloader controlla se una chiave è già presente nell'eFuse:
  - Se è presente, il processo di generazione della chiave viene saltato e viene usata la chiave esistente.

- Se non è presente, viene generata casualmente una nuova chiave di crittografia AES-256 bit e viene scritta nell'eFuse `flash_encryption`.

L'intero processo di crittografia avviene via hardware e la chiave non è accessibile dal software.

4. Il *flash encryption block* crittografa tutti i dati nella memoria flash, cioè il second stage bootloader, le applicazioni e tutte le partizioni etichettate con la flag `encrypted` nella partition table.
5. Il second stage bootloader imposta il primo bit disponibile di `FLASH_CRYPT_CNT` a 1 per indicare che i dati nella memoria flash sono crittografati.
6. In modalità `development`, il second stage bootloader abilita soltanto gli eFuse `DISABLE_DL_ENCRYPT` e `DISABLE_DL_CACHE` per permettere al bootloader in modalità Firmware Download di flashare binari crittografati. Inoltre i bit dell'eFuse `FLASH_CRYPT_CNT` non sono protetti in scrittura.
7. In modalità `release`, il second stage abilita i bit degli eFuse `DISABLE_DL_ENCRYPT`, `DISABLE_DL_DECRYPT` e `DISABLE_DL_CACHE` per impedire che il bootloader decrittografi i dati durante la modalità Firmware Download. Inoltre i bit dell'eFuse `FLASH_CRYPT_CNT` sono protetti in scrittura.
8. Il dispositivo viene riavviato per eseguire l'immagine crittografata. Il second stage bootloader chiama il *flash decryption block* per decrittografare i dati della memoria flash e caricare i contenuti decrittografati nella memoria IRAM (Instruction RAM).

### 2.4.3 Modalità di sviluppo e di rilascio

Durante la fase di sviluppo è necessario poter flashare più volte il dispositivo, con diverse immagini non crittografate per testare il processo di crittografia, di conseguenza la modalità Firmware Download deve essere attivata. Questa modalità però dovrebbe essere disabilitata nella versione finale del prodotto per impedire ulteriore accesso alla memoria flash. Per questo motivo esistono due modalità di funzionamento della Flash Encryption:

- **Development mode:** In questa modalità è possibile flashare nuove immagini non crittografate sul dispositivo e il bootloader le crittograferà utilizzando la chiave memorizzata nell'hardware. Questo però permette, indirettamente, di leggere i contenuti in chiaro dell'immagine in memoria flash.
- **Release mode:** In questa modalità non è più possibile flashare nuove immagini sul dispositivo senza conoscere la chiave di crittografia.

## 3 Implementazione

### 3.1 Aggiornamenti OTA

#### 3.1.1 Configurazione del progetto

Per abilitare gli aggiornamenti OTA è stato necessario configurare il progetto ESP-IDF con le seguenti componenti:

- **Partition table e memoria flash:** è stato creato il file `sdkconfig.defaults` che contiene la configurazione di default del progetto. All'interno è stato definito l'utilizzo della partition table con due partizioni OTA:

```
1 CONFIG_PARTITION_TABLE_TWO_OTA=y
```

e sono state impostate le dimensioni della memoria flash a 4MB:

```
1 CONFIG_ESPTOOLPY_FLASHSIZE_4MB=y
```

- **Server HTTP:** per permettere il download del nuovo firmware è stato creato un server HTTP in `ota_https_server` che rende disponibile il file binario del firmware aggiornato. La cartella che il server condivide di default è `ota_https_server/firmware` e di conseguenza il file binario del firmware deve essere copiato (o fare un link simbolico) in questa cartella per essere accessibile dal dispositivo.

Se si vuole aprire il server in modalità HTTPS, allora è necessario generare un certificato SSL per permettere connessioni sicure. Il certificato è si può generare eseguendo il seguente comando in una qualsiasi cartella, in questo caso è stata usata la cartella `ota_https_server/certificates`:

```
1 cd ota_https_server/certificates
2 openssl req -x509 -newkey rsa:2048 -keyout ca_key.pem -out
   ca_cert.pem -days 365 -nodes
```

Questo comando genera due file:

- `ca_key.pem`: chiave privata del certificato
- `ca_cert.pem`: certificato SSL pubblico

**Nota:** durante la creazione del certificato nel campo **Common Name (CN)** deve essere inserito il nome host del server. Se il server viene eseguito in locale il campo deve essere impostato con l'indirizzo IP del server e bisogna disabilitare il controllo del CN nel `menuconfig` del progetto.

Una volta generato il certificato bisogna flashare il file `ca_cert.pem` sul dispositivo in modo che possa verificare l'autenticità del server. Per fare ciò bisogna copiare il file nella cartella `server_certs/` del progetto (la working directory è ancora `ota_https_server/certificates`):

```
1 cp ca_cert.pem ../../server_certs
```

Inoltre è necessario modificare il file `main/CMakeLists.txt` per includere il certificato nel firmware con il seguente contenuto:

```

1 idf_build_get_property(project_dir PROJECT_DIR)
2 idf_component_register(SRCS "main.c" INCLUDE_DIRS ".")
3   EMBED_TXTFILES ${project_dir}/server_certs/ca_cert.pem)

```

Una volta configurato il tutto, il server può essere avviato eseguendo il comando:

- Server HTTP senza certificato SSL:

```
1 cargo run --release
```

- Server HTTPS con certificato SSL:

```
1 cargo run --release -- -c <CERT_DIR>
```

Dove:

- <CERT\_DIR> è cartella che contiene il certificato SSL del server. Se non viene specificata viene usata la cartella corrente.

È possibile configurare altri parametri che sono elencati eseguendo il comando:

```
1 cargo run --release -- --help
```

Se tutto è andato a buon fine l'output del server sarà simile al seguente:

```

1 $ cargo run --release -- -c certificates
2   Finished 'release' profile [optimized] target(s) in 0.11s
3   Running 'target/release/ota_https_server -c certificates'
4 2025-11-28T14:50:04.418063Z INFO ota_https_server: --cert-dir
   provided, starting in HTTPS mode.
5 2025-11-28T14:50:04.418077Z INFO ota_https_server: Using
   certificate: "/home/fabio/dev/esp32/esp32_secure_ota/
   ota_https_server/certificates/ca_cert.pem"
6 2025-11-28T14:50:04.418082Z INFO ota_https_server: Using
   private key: "/home/fabio/dev/esp32/esp32_secure_ota/
   ota_https_server/certificates/ca_key.pem"
7 2025-11-28T14:50:04.418199Z INFO ota_https_server: Starting
   HTTPS server on https://0.0.0.0:8070/ serving files from
   "/home/fabio/dev/esp32/esp32_secure_ota/ota_https_server/
   firmware"
8 2025-11-28T14:50:04.418205Z INFO ota_https_server: Your ESP32
   should connect to: https://<your_ipv4>:8070/<bin_name>

```

- **Supporto del versionamento:** per tenere traccia delle versioni del firmware è stato aggiunto il file `version.txt` nella cartella principale del progetto che contiene il numero di versione corrente. Questo numero viene inserito nel file binario del firmware durante la compilazione e può essere letto dall'applicazione per confrontare la versione corrente con quella disponibile sul server ed evitare aggiornamenti non necessari.
- **Accesso Wi-Fi:** per connettere il dispositivo alla rete Wi-Fi è stata implementata la logica di connessione nel file `main/wifi.c`.
- **Configurazione Wi-Fi e server HTTP:** per fornire le credenziali di accesso alla rete Wi-Fi e l'indirizzo del server HTTP è stata creata una sezione nel menu di configurazione del progetto (`menuconfig`), nel file `main/Kconfig.projbuild`. Per accedere al `menuconfig` bisogna eseguire il comando:

```
1 idf.py menuconfig
```

e navigare fino alla sezione `Over The Air Updates configuration` dove è possibile impostare:

– **Wifi configuration**

- \* **SSID**: nome della rete Wi-Fi
- \* **Password**: password della rete Wi-Fi
- \* **Retry interval**: intervallo (in secondi) tra un tentativo di connessione e l'altro in caso di disconnessione
- \* **WPA3 SAE mode selection**: modalità di autenticazione WPA3
- \* **Password identifier**: identificatore della password WPA3
- \* **WiFi Scan auth mode threshold**: modalità di autenticazione minima accettata durante la scansione delle reti

– **Upgrade server**

- \* **Firmware upgrade URL**: URL del server HTTP che ospita il file binario del firmware. Deve essere nel formato:

```
1 https://<host-ip-address>:<host-port>/<firmware-image-  
   filename>
```

dove:

- **<host-ip-address>**: hostname o indirizzo IP del server HTTP.
- **<host-port>**: porta su cui è in esecuzione il server HTTP.
- **<firmware-image-filename>**: nome del file binario del firmware. **Deve coincidere con il nome del file messo a disposizione dal server.**
- \* **Skip server certificate CN field check**: se abilitato, il dispositivo non verificherà il campo **Common Name (CN)** del certificato SSL. Utile per testare il server in locale senza dover generare un certificato con l'indirizzo IP come CN.
- \* **Skip firmware version check**: se abilitato, il dispositivo non confronterà la versione del firmware corrente con quella disponibile sul server prima di effettuare l'aggiornamento OTA.
- \* **OTA recieve timeout**: tempo massimo (in millisecondi) per ricevere una risposta dal server HTTP durante l'aggiornamento OTA.
- \* **OTA retry interval**: intervallo di tempo (in secondi) tra un controllo e l'altro della disponibilità di un nuovo firmware sul server.

### 3.1.2 Connessione Wi-Fi

La connessione alla rete Wi-Fi viene gestita nel file `main/wifi.c` che fornisce la funzione `connect_wifi()`. Questa funzione gestisce la connessione alla rete Wi-Fi utilizzando le credenziali fornite nel `menuconfig` e la riconnessione in caso di disconnessione. La funzione esegue i seguenti passaggi:

1. Inizializza il driver Wi-Fi

2. Chiama un handler per gestire gli eventi di connessione e disconnessione dalla rete Wi-Fi
3. Chiama un handler per gestire l'acquisizione dell'indirizzo IP
4. Imposta il device in modalità stazione Wi-Fi con la configurazione fornita dal menuconfig
5. Avvia la connessione alla rete Wi-Fi
6. Attende fino a quando il dispositivo non si connette alla rete
7. Registra degli handler per gestire gli eventi di disconnessione e riconnessione. Se avviene una disconnessione si tenta di riconnettersi con un intervallo impostabile tramite il menuconfig
8. Restituisce lo stato della connessione (successo o fallimento)

Gli handler rimangono attivi per tutta la durata dell'applicazione per gestire eventuali disconnessioni e tentativi di riconnessione anche dopo che la funzione `connect_wifi()` ha restituito il controllo all'applicazione principale.

### 3.1.3 Aggiornamento OTA

L'aggiornamento OTA viene gestito nel file `main/ota.c` che fornisce le funzioni `download_new_firmware()` e `diagnose_new_firmware()`. La funzione `download_new_firmware()` connette il dispositivo al server HTTP e scarica il nuovo firmware, se disponibile, e lo imposta come nuova applicazione di boot. I passi eseguiti sono i seguenti:

1. Recupera le partizioni di boot e OTA correnti
2. Si connette al server HTTP utilizzando l'URL fornito nel menuconfig
3. Recupera la partizione OTA non attualmente in uso per il boot
4. Legge il file binario del firmware dal server HTTP
5. Confronta la versione del firmware corrente con quella scaricata
  - Se l'aggiornamento non è necessario viene eseguito un loop infinito che attende un reset del dispositivo per ritentare l'aggiornamento
6. Inizializza l'aggiornamento OTA
7. Scrive i dati del firmware scaricato nella partizione OTA non in uso
8. Controlla che tutti i dati siano stati scritti correttamente
9. Disattiva l'aggiornamento OTA
10. Imposta la partizione OTA appena scritta come partizione di boot
11. Riavvia il dispositivo per eseguire il nuovo firmware

Questi passi vengono ripetuti con un intervallo di tempo impostabile tramite il menuconfig per controllare periodicamente la disponibilità di un nuovo firmware. La funzione `diagnose_new_firmware()` permette di verificare se il nuovo firmware funziona correttamente e, in caso negativo, effettua il rollback alla versione precedente. I passi eseguiti sono i seguenti:

1. Controlla se l'applicazione corrente è al primo avvio dopo un aggiornamento OTA
2. Esegue la funzione di diagnosi del firmware (in questo caso un semplice delay di 5 secondi che simula un controllo)
  - Se la diagnosi ha esito positivo, conferma che il nuovo firmware è valido e cancella il rollback
  - Se la diagnosi ha esito negativo, marca il firmware come non valido ed esegue il rollback alla versione precedente

#### 3.1.4 Applicazione principale

L'applicazione principale si trova nel file `main/main.c` e utilizza le funzioni definite nei file `wifi.c` e `ota.c` per connettere il dispositivo alla rete Wi-Fi e scaricare il nuovo firmware. I passi eseguiti sono i seguenti:

1. Controlla il nuovo firmware tramite la funzione `diagnose_new_firmware()`
2. Si connette alla rete Wi-Fi tramite la funzione `connect_wifi()`
3. Crea una task per gestire l'aggiornamento OTA chiamando la funzione `download_new_firmware()`
4. Crea una task per eseguire l'applicazione principale (in questo caso un semplice loop che stampa un messaggio ogni 5 secondi)

## 3.2 Flash Encryption

### 3.2.1 Configurazione del progetto

Prima di attivare la Flash Encryption bisogna assicurarsi di avere i valori default degli eFuse relativi alla Flash Encryption. I passi per attivare la Flash Encryption sono i seguenti dal menuconfig:

1. Attivare la Flash Encryption all'avvio in:

```
1 Security features > Enable Flash Encryption on boot
```

2. Selezionare la modalità di Flash Encryption in:

```
1 Security features > Enable usage mode
```

Qui si può scegliere tra:

- **Development mode:** permette di flashare nuove immagini non crittografate sul dispositivo per testare il processo di crittografia.
- **Release mode:** non permette più di flashare nuove immagini sul dispositivo senza conoscere la chiave di crittografia.



3. Disattivare NVS Encryption in:

```
1 Component config > NVS > Enable NVS encryption
2
```

Per questo progetto non è necessario usare anche la NVS Encryption, se si volesse attivare servirebbe definire una partition table personalizzata con una partizione

```
1 # Name,      Type, SubType, Offset,  Size, Flags
2 ...
3 nvs_keys, data, nvs_keys,,          0x1000, encrypted
4 ...
```

4. Selezionare la modalità di UART Download (disponibile solo dopo aver impostato la versione minima di ESP32 a 3 in:

```
1 Component config > Hardware Settings > Chip revision > Minimum
   Supported ESP32 Revision (Rev v3.x)
```

). La modalità di UART Download può essere configurata in:

```
1 Security features > UART ROM Download mode
```

È consigliato disattivare la modalità di UART Download in Release mode per impedire l'accesso non autorizzato alla memoria flash. Per il testing, quindi in modalità Development, è meglio lasciarla abilitata. Una volta disabilitata non sarà più possibile riabilitarla.

5. Abilitare la Flash Encryption aumenta la dimensione del bootloader, quindi bisogna aumentare l'offset della partition table:

```
1 Partition Table > Offset of partition table
```

Per questo progetto è stato impostato a 0x10000 (64KB).

6. Salvare e uscire dal menuconfig.

7. Eseguire il seguente comando per buildare e flashare l'immagine sul dispositivo:

```
1 idf.py flash monitor
```

Questo comando scrive sulla memoria flash immagini non crittografate, cioè:

- il bootloader
- la partition table
- l'applicazione principale

Una volta che la fase di flash è completata il dispositivo si riavvia e all'avvio successivo il second stage bootloader crittografa:

- il second stage bootloader
- partizioni di tipo app
- partizioni con la flag **encrypted** nella partition table

e alla fine si riavvia nuovamente per eseguire l'immagine crittografata. Agli avvi successivi il bootloader decrittografa automaticamente i dati della memoria flash prima di eseguirli.

Questo comando non include alcun file definito dall'utente e quindi i file da includere nella memoria flash devono essere scritti manualmente. Se non viene fatto prima che il bootloader esegua la crittografia, questi file dovranno essere crittografati separatamente prima di scriverli nella memoria flash. Nel caso di questo progetto tutti i file aggiuntivi sono inclusi automaticamente durante la build dell'immagine del firmware e non c'è bisogno di fare alcun passaggio extra.

Dopo aver completato questi passaggi, la Flash Encryption è attiva e da questo momento in poi per aggiornare l'applicazione tramite USB, se Flash Encryption è in modalità Development con la modalità di UART Download abilitata, bisogna prima crittografarla usando il seguente comando:

```
1 idf.py encrypted-app-flash monitor
```

Per aggiornare tutte le partizioni crittografandole si usa il comando:

```
1 idf.py encrypted-flash monitor
```

Altrimenti, tramite aggiornamento OTA, il dispositivo può scaricare il firmware in chiaro e crittografarlo localmente prima di scriverlo in memoria flash.

### 3.3 Secure boot

#### 3.3.1 Configurazione del progetto

Una volta attivata la Flash Encryption si può attivare il Secure Boot v2. I passi per attivare il Secure Boot v2 sono i seguenti:

1. La versione 2 del Secure Boot è disponibile soltanto per chip ESP32 con versione v3.0 o maggiore. Bisogna cambiare la versione minima del chip supportata dal progetto dal menuconfig:

```
1 Component config > Hardware Settings > Chip revision > Minimum  
   Supported ESP32 Revision (Rev v3.x)
```

2. Generare una chiave RSA-3072 per firmare le immagini del firmware. Questo può essere fatto eseguendo il seguente comando nella cartella principale del progetto:

```
1 idf.py secure-generate-signing-key --version 2 --scheme  
   rsa3072 <file_name>
```

**Nota:** la chiave generata deve essere sempre mantenuta segreta.

3. Abilitare il Secure Boot v2 in:

```
1 Security features > Enable hardware Secure Boot in bootloader  
2
```

4. Specificare il percorso, relativo alla directory del progetto, della chiave generata nel passo 2 in:

5. Selezionare la modalità di UART Download in:

```
1 Security features > UART ROM Download mode
```

È consigliato disattivare la modalità di UART Download in production, ma per il testing è meglio lasciarla abilitata. Una volta disabilitata non sarà più possibile riabilitarla.

6. Salvare e uscire dal menuconfig.

7. Eseguire il comando

```
1 idf.py bootloader
```

per buildare un bootloader firmato per il Secure Boot. L'output della build includerà un comando per flashare il bootloader firmato sulla memoria flash usando:

```
1 esptool.py write_flash --encrypt
```

**Nota:** siccome la flash encryption dovrebbe essere già stata attivata, bisogna usare l'opzione `--encrypt` per flashare il bootloader firmato.

8. Flashare il bootloader firmato sulla memoria flash eseguendo il comando mostrato nell'output della build del bootloader.

9. Eseguire il comando:

```
1 idf.py encrypted-flash monitor
```

per buildare, flashare e crittografare (per Flash Encryption) la partition table e l'applicazione principale. L'immagine verrà firmata utilizzando la chiave specificata nel passo 4. **Nota:** il comando `idf.py encrypted-flash` non include il bootloader quando il Secure Boot è abilitato, quindi bisogna flasharlo separatamente come descritto nel passo precedente.

10. Riavviare il dispositivo per eseguire il bootloader firmato. Il bootloader abiliterà il Secure Boot sul chip, verificherà la firma dell'immagine dell'applicazione e la eseguirà. Eventuali errori verranno mostrati nella console e il Secure Boot non verrà abilitato finché non vengono flashate immagini valide per evitare di bloccare il dispositivo.
11. Da questo momento in poi, ad ogni avvio il Secure Boot verificherà che il bootloader non sia stato modificato e che l'immagine dell'applicazione sia valida prima di eseguirla.

## 4 Analisi dei risultati

### 4.1 Verifica dello stato di sicurezza

Controllare lo stato di attivazione delle features di sicurezza può essere fatto in due modi:

- **Metodo hardware:** leggendo lo stato degli eFuse del dispositivo
- **Metodo software:** controllando i log della console all'avvio del dispositivo

### 4.1.1 Controllo dello stato degli eFuse

È possibile usare lo strumento `espefuse.py` per leggere lo stato degli eFuse del dispositivo. Eseguendo il seguente comando (il dispositivo deve essere collegato):

```
1 espefuse.py --port <PORT> summary
```

l'output dovrà essere simile al seguente, dove si può notare che la Flash Encryption è abilitata (con un numero dispari di bit impostati in `FLASH_CRYPT_CNT`) e il Secure Boot v2 è abilitato (con il bit `ABS_DONE_1` impostato a 1):

```
1 === Run "summary" command ===
2 EFUSE_NAME (Block) Description = [Value] [Readable/Writeable] (Hex Value)
3 -----
4
5 ...
6
7 Flash fuses:
8 FLASH_CRYPT_CNT (BLOCK0) Flash encryption is enabled = 1 R/W (0b0000001)
9 if this field has an odd
10 number of bits set
11 FLASH_CRYPT_CONFIG (BLOCK0) Flash encryption config = 15 R/W (0xf)
12 (key tweak bits)
13
14 Security fuses:
15 UART_DOWNLOAD_DIS (BLOCK0) Disable UART download mode. = False R/W (0b0)
16 Valid for ESP32 V3 and
17 newer; only
18 ABS_DONE_0 (BLOCK0) Secure boot V1 is enabled = False R/W (0b0)
19 for bootloader image
20 ABS_DONE_1 (BLOCK0) Secure boot V2 is enabled = True R/W (0b1)
21 for bootloader image
22 DISABLE_DL_ENCRYPT (BLOCK0) Disable flash encryption in = False R/W (0b0)
23 UART bootloader
24 DISABLE_DL_DECRYPT (BLOCK0) Disable flash decryption in = True R/W (0b1)
25 UART bootloader
26 KEY_STATUS (BLOCK0) Usage of efuse block 3 = False R/W (0b0)
27 (reserved)
28 SECURE_VERSION (BLOCK3) Secure version for = 0 R/W (0x00000000)
29 anti-rollback
30 BLOCK1 (BLOCK1) Flash encryption key
31 = ?????????????????????????????????????????????????????????????? -/-
32 BLOCK2 (BLOCK2) Security boot key
33 = ***** R/-
34 BLOCK3 (BLOCK3) Variable Block 3
35 = 0000000000000000000000000000000000000000000000000000000000000000 R/W
36
37 ...
```

### 4.1.2 Controllo dei log della console

Dopo aver eseguito tutti i passaggi per attivare la Flash Encryption e il Secure Boot v2, all'avvio del dispositivo verranno mostrati dei log simili ai seguenti che confermano che entrambe le features di sicurezza sono attive:

```
1 ...
2
3 I (473) esp_image: Verifying image signature...
4 I (475) secure_boot_v2: Verifying with RSA-PSS...
5 I (483) secure_boot_v2: Signature verified successfully!
6 I (496) boot: Loaded app from partition at offset 0x120000
7 I (496) secure_boot_v2: enabling secure boot v2...
8 I (496) secure_boot_v2: secure boot v2 is already enabled, continuing..
9 I (502) boot: Checking flash encryption...
10 I (506) flash_encrypt: flash encryption is enabled (3 plaintext flashes left)
11
12 ...
```

## 4.2 Test di aggiornamento OTA

Per testare il corretto funzionamento dell'aggiornamento OTA sicuro basta incrementare il numero della versione nel file `version.txt` dopo aver modificato il codice (o anche senza modifiche) e ricompilare il progetto eseguendo il comando:

```
1 idf.py build
```

Il compilatore include automaticamente il signature block nell'immagine del firmware e lo firma utilizzando la chiave specificata nel menuconfig. Una volta che la build è completata, il file binario del firmware aggiornato si trova in `build/esp32.secure_ota.bin` e deve essere copiato nella cartella `ota_https_server/firmware` per essere reso disponibile dal server HTTP. In alternativa si può creare un link simbolico. Quando il dispositivo esegue il controllo periodico del server HTTP, scarica il nuovo firmware e se la versione è maggiore di quella corrente, procede con l'aggiornamento OTA e si riavvia per eseguire il nuovo firmware. L'output sarà simile al seguente:

```
1 ...
2 I (491592) APP: Hello, from firmware version: 22!
3 I (492622) OTA: Attempting to download new firmware...
4 I (492622) OTA: Running partition type 0 subtype 0 (offset 0x00020000)
5 I (493212) OTA: New firmware version: 23
6 I (493222) OTA: Running firmware version: 22
7 I (493222) OTA: Writing to partition subtype 16 at offset 0x120000
8 I (493222) OTA: esp_ota_begin succeeded
9 I (493222) OTA: Updating firmware...
10 I (514332) OTA: Connection closed
11 I (514332) OTA: Total Write binary data length: 921600
12 ...
13 I (515032) esp_image: Verifying image signature...
14 I (515032) secure_boot_v2: Take trusted digest key(s) from eFuse block(s)
15 I (515032) secure_boot_v2: #0 app key digest == #0 trusted key digest
16 I (515042) secure_boot_v2: Verifying with RSA-PSS...
17 I (515102) secure_boot_v2_rsa: Signature verified successfully!
18 I (515172) OTA: Prepare to restart system!
19 I (515172) OTA: Waiting for network stack to clean up...
20 ...
```

Al riavvio, il bootloader verifica la firma digitale della nuova immagine crittografata e, se valida, avvia l'applicazione aggiornata. L'output sarà simile al seguente:

```
1 ...
2 I (473) esp_image: Verifying image signature...
3 I (475) secure_boot_v2: Verifying with RSA-PSS...
4 I (483) secure_boot_v2: Signature verified successfully!
5 I (496) boot: Loaded app from partition at offset 0x120000
6 ...
7 I (1239) WIFI: WIFI station initialization complete
8 I (1239) WIFI: Connecting to AP
9 I (2409) WIFI: Station IP: 192.168.1.1
10 I (2409) WIFI: Connected to AP
11 I (2409) OTA: Starting new firmware download task
12 I (2409) OTA: Attempting to download new firmware...
13 I (2419) OTA: Running partition type 0 subtype 16 (offset 0x00120000)
14 I (2439) APP: Hello, from firmware version: 23!
15 I (2739) OTA: New firmware version: 23
16 I (2739) OTA: Running firmware version: 23
17 W (2739) OTA: Current running version is the same as a new.
18     The update will not be made.
19 I (2739) OTA: No new firmware version available. Retrying in 30s...
20 ...
```

## 5 Conclusioni

Il progetto ha portato alla realizzazione e alla validazione di un sistema embedded basato su ESP32 capace di eseguire aggiornamenti firmware da remoto (OTA) in modo sicuro.

Le principali considerazioni riguardo le funzionalità implementate sono:

- **Sicurezza Hardware:** L'abilitazione del Secure Boot v2 e della Flash Encryption garantiscono una sicurezza contro gli attacchi fisici o a tentativi di esecuzione di codice non autorizzato.
- **Affidabilità dell'aggiornamento:** L'implementazione del meccanismo di rollback garantisce la continuità di servizio. Anche in caso di firmware corrotto o malfunzionante, il sistema è in grado di rilevare il problema e tornare automaticamente alla versione precedente, evitando che il dispositivo diventi inutilizzabile ("bricked").
- **Gestione degli eFuse:** L'aspetto più complesso dell'implementazione è l'irreversibilità di alcune operazioni. La gestione delle chiavi crittografiche e la configurazione degli eFuse richiedono particolare attenzione, perchè la perdita della chiave privata o un errore nella configurazione delle features di sicurezza possono rendere il chip permanentemente inutilizzabile.

## Bibliografia

- [1] Espressif Systems. *Documentation of QEMU Fork with Espressif patches*. URL: <https://github.com/espressif/esp-toolchain-docs/blob/main/qemu/esp32/README.md>. (accessed: 05.11.2025).
- [2] Espressif Systems. *eFuse*. URL: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/efuse.html>. (accessed: 04.11.2025).
- [3] Espressif Systems. *Enable Flash Encryption and Secure Boot Externally*. URL: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/security/security-features-enablement-workflows.html#enable-flash-encryption-and-secure-boot-v2-externally>. (accessed: 25.11.2025).
- [4] Espressif Systems. *Flash Encryption*. URL: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/security/flash-encryption.html>. (accessed: 10.11.2025).
- [5] Espressif Systems. *Flash Encryption Example*. URL: [https://github.com/espressif/esp-idf/tree/v5.5.1/examples/security/flash\\_encryption](https://github.com/espressif/esp-idf/tree/v5.5.1/examples/security/flash_encryption). (accessed: 25.11.2025).
- [6] Espressif Systems. *Fork of QEMU with Espressif patches*. URL: <https://github.com/espressif/qemu>. (accessed: 11.11.2025).
- [7] Espressif Systems. *Over The Air Examples*. URL: <https://github.com/espressif/esp-idf/tree/master/examples/system/ota>. (accessed: 27.10.2025).

- [8] Espressif Systems. *Over The Air Updates (OTA)*. URL: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/ota.html>. (accessed: 27.10.2025).
- [9] Espressif Systems. *Secure Boot v2*. URL: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/security/secure-boot-v2.html#>. (accessed: 03.11.2025).
- [10] Espressif Systems. *Security Features Example*. URL: [https://github.com/espressif/esp-idf/tree/v5.5.1/examples/security/security\\_features\\_app#enable-security-features-with-help-of-qemu](https://github.com/espressif/esp-idf/tree/v5.5.1/examples/security/security_features_app#enable-security-features-with-help-of-qemu). (accessed: 26.11.2025).